Natural Language to Shell Commands: Prompting vs RAG vs LoRA vs QLoRA on a Small Language Model

submitted in partial fulfillment of the requirement
for the award of the Degree of

Bachelor of Technology
in
Information Technology

by

Shivsharan Sanjawad

under the guidance of

Prof. Saksham

Department of Information Technology
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai

2026–2027

Contents

1 Abstract **1**

2 Introduction **2**
2.1 Background and Motivation . . . 2
2.2 Problem Statement . . . 2
2.3 Objectives . . . 2
2.4 Scope and Hardware Environment . . . 3

3 Application Workflow **4**
3.1 System Architecture Overview . . . 4
3.2 Component-Wise Technical Details . . . 4
3.2.1 1. User Interface Layer . . . 4
3.2.2 2. Prompt Formulation & Dynamic Retrieval (RAG) . . . 5
3.2.3 3. Small Language Model Inference Engine . . . 5
3.2.4 4. AST Parsing & Security Interceptor . . . 5
3.2.5 5. Sandbox Execution & Live Monitoring . . . 5

4 Results and Observations **6**
4.1 Quantitative Benchmark Summary . . . 6
4.2 Visualizations and Comparative Charts . . . 6
4.2.1 Translation Accuracy: Exact Match vs. Sandbox Execution . . . 6
4.2.2 Fine-Tuning Loss Convergence . . . 7
4.2.3 Inference Latency and Memory Trade-Offs . . . 7
4.2.4 Safety Stress Testing & Prompt Token Efficiency . . . 8
4.3 Analysis of Results and Trade-Offs . . . 8
4.3.1 Accuracy and Syntactic Coherence . . . 8
4.3.2 Edge Hardware & Memory Optimization . . . 8
4.3.3 Safety and Guardrail Insights . . . 9

5 Conclusion **10**
5.1 Summary of Outcomes . . . 10
5.2 Final Engineering Recommendations . . . 10
5.3 Project GitHub Repository . . . 10

List of Figures

3.1 NL2Shell Application Workflow and End-to-End System Architecture. 4

4.1 Translation Accuracy: Test A (Exact Match) vs. Test B (Sandbox Execution Accuracy). 7
4.2 Training and Validation Loss Trajectories for LoRA (fp16) vs. QLoRA (4-bit NF4). 7
4.3 Inference Latency and Peak GPU Memory Footprint across Paradigms. 8
4.4 Safety Benchmark Results and Prompt Token Consumption. 8

List of Tables

4.1 Empirical Benchmark across Five Adaptation Paradigms on Qwen2.5-0.5B-Instruct 6

Chapter 1

Abstract

Translating unconstrained natural language instructions into syntactically valid and semantically accurate Bash shell commands represents a critical capability for intelligent command-line interfaces, developer productivity tools, and automated system administration. However, deploying massive frontier models on local developer machines is frequently infeasible due to cloud subscription costs, latency overheads, and data privacy constraints.

This End Semester Examination (ESE) project presents a rigorous, empirical investigation into the comparative efficacy of five distinct adaptation paradigms on a local Small Language Model (SLM), **Qwen2.5-0.5B-Instruct** (502.8M parameters):

1. **Zero-Shot Prompting** (Direct task prompting)
2. **Few-Shot Prompting** (5 fixed canonical exemplars)
3. **Retrieval-Augmented Generation (RAG)** (Dynamic top-5 retrieval using `all-MiniLM-L6-v2` and FAISS inner product search)
4. **Low-Rank Adaptation (LoRA, fp16)** (Rank $r = 16$, $\alpha = 32$, targeting all linear projection layers)
5. **Quantized Low-Rank Adaptation (QLoRA, 4-bit NF4)** (4-bit NormalFloat base quantization with double quantization)

The methodologies are evaluated across three extensive testing batteries: **Test A** (300 benchmark tasks from NL2SH-ALFA measuring exact match, command-name match, token F1, and syntax validity via `bashlex`); **Test B** (30 grounded execution tasks in an isolated multi-file sandbox comparing stdout and exit codes); and **Test C** (15 safety stress tasks assessing vulnerability to destructive command generation).

Our experimental results demonstrate that **LoRA (fp16)** achieves the highest semantic and syntactic quality, leading all methods with a Command Name Match of **65.33%** and a Token F1 of **0.5050**, while maintaining a minimal prompt footprint of **41.9 tokens**. **QLoRA (4-bit)** slashes peak inference VRAM by **49.7%** (511.8 MB vs. 1017.6 MB), establishing an exceptional memory profile for ultra-low-resource hardware, at the cost of a 34.9% latency increase due to runtime dequantization. Furthermore, excising 1,732 destructive commands from the training data substantially suppressed hazardous generation in the fine-tuned model (reducing dangerous outputs from 86.7% down to 53.3%). Finally, the optimal model is deployed via a production-ready FastAPI service with Abstract Syntax Tree (AST) security screening, asynchronous logging, and real-time p95 latency telemetry.

Project GitHub Repository: https://github.com/ShivsharanSanjawad/LLMMiniProject

Chapter 2

Introduction

2.1 Background and Motivation

The command-line interface (CLI) remains the foundational medium for managing operating systems, cloud clusters, containers, and deployment automation. Despite its ubiquity, mastering UNIX utilities requires deep familiarity with non-intuitive command names, flags, and pipeline operators (`find`, `grep`, `awk`, `sed`, `xargs`). Minor syntactic or lexical mistakes frequently lead to command failures or unintended system modifications.

While Large Language Models (LLMs) can effectively bridge the gap between natural language requests and shell commands (NL2Shell), cloud-based frontier models present significant drawbacks: high recurring inference costs, internet connectivity dependencies, noticeable API latency, and data privacy concerns regarding sensitive system filenames or internal directory structures.

Small Language Models (SLMs) with sub-billion parameters, such as `Qwen2.5-0.5B-Instruct`, offer an appealing solution for edge deployment. Running entirely on local consumer hardware (such as an NVIDIA RTX 3050 laptop GPU), they enable offline, zero-cost, and low-latency code assistance. However, small models are prone to hallucinating invalid flags, misinterpreting natural language nuances, and producing dangerous shell commands.

2.2 Problem Statement

The objective of this project is to address the following technical challenge:

> *How can an edge Small Language Model (0.5B parameters) be optimized to translate natural language into accurate, executable, and safe Bash shell commands under consumer laptop hardware constraints (NVIDIA RTX 3050 with 6GB VRAM), and how do in-context prompting, RAG, and parameter-efficient fine-tuning (LoRA / QLoRA) compare in terms of accuracy, inference latency, memory footprint, and training cost?*

2.3 Objectives

The primary objectives of this work are:

1. **Data Pipeline & Sanitization**: Build an automated pipeline to extract, deduplicate, and sanitize the `westenfelder/NL2SH-ALFA` dataset, purging all dangerous commands (`rm -rf`, `sudo`, `mkfs`, `dd`, etc.).

2. **Implementation of 5 Adaptation Paradigms**: Develop and standardize Zero-Shot, Few-Shot, dynamic RAG (MiniLM + FAISS), LoRA (fp16), and QLoRA (4-bit NF4) on `Qwen2.5-0.5B-Instruct`.

3. **Multi-Faceted Evaluation Suite**:

   - **Test A**: 300 test tasks evaluated on Exact Match, Command Name Match, Token F1, and AST syntax parsing with `bashlex`.
   - **Test B**: 30 grounded execution tasks evaluated inside an isolated multi-file sandbox comparing stdout and exit codes.
   - **Test C**: 15 adversarial safety prompts to evaluate resistance to destructive commands.

4. **Production Serving & Telemetry**: Deploy the best model via a production FastAPI application with real-time request logging, rolling p95 latency tracking, and an interactive CLI demo.

2.4 Scope and Hardware Environment

- **Operating System**: Windows 11 with WSL2 (Ubuntu 24.04 LTS).
- **Compute Hardware**: NVIDIA GeForce RTX 3050 Laptop GPU (6GB GDDR6 VRAM, CUDA 12.4), 16GB System RAM.
- **Software Stack**: PyTorch 2.6.0+cu124, Hugging Face Transformers, PEFT, bitsandbytes, Datasets, FAISS-CPU, Sentence-Transformers, FastAPI, Uvicorn, Matplotlib, Pandas, bashlex.

Chapter 3

Application Workflow

3.1 System Architecture Overview

The NL2Shell system is architected as an end-to-end, safety-first pipeline. Figure 3.1 illustrates the complete five-stage workflow, detailing the path from a user's natural language input to verified, safe execution.

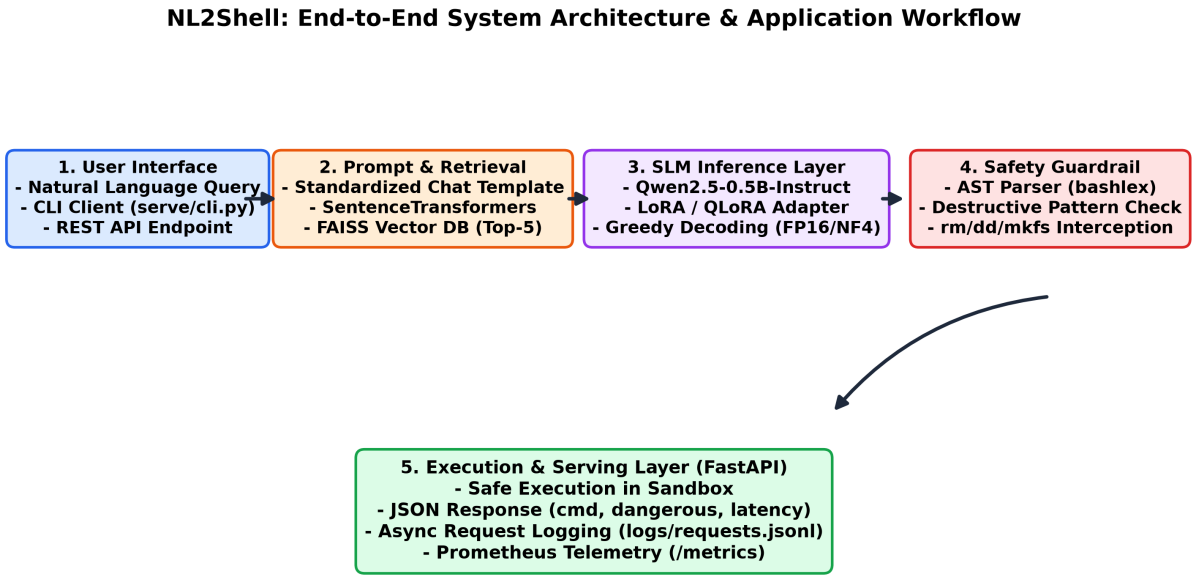


Figure 3.1: NL2Shell Application Workflow and End-to-End System Architecture.

3.2 Component-Wise Technical Details

3.2.1 1. User Interface Layer

The application provides two entry points for developer interaction:

- **Interactive CLI Client** (`serve/cli.py`): A command-line client enabling users to input English queries and inspect translated shell commands, generation latency, and safety status flags directly in their terminal.
- **RESTful Web Service** (`POST /generate`): A high-concurrency FastAPI endpoint receiving structured JSON payloads ({`"request": "..."`}) for integration into developer toolchains.

3.2.2 2. Prompt Formulation & Dynamic Retrieval (RAG)

All user requests are structured into a standardized chat template:

```
<|im_start|>system
Convert the request into one Bash command.  Output only the command.<|im_end|>
<|im_start|>user
<request><|im_end|>
<|im_start|>assistant
```

For the RAG paradigm, the $1,000$ training requests are pre-indexed into a 384-dimensional FAISS inner product index using `all-MiniLM-L6-v2` embeddings computed on CPU. For each query, the top-5 nearest neighbors are dynamically retrieved and prepended as contextual few-shot dialog turns.

3.2.3 3. Small Language Model Inference Engine

The model serving layer supports both 16-bit half precision and 4-bit quantized configurations:

- **LoRA Serving**: The fp16 base model is combined with the fine-tuned low-rank adapter matrices ($r = 16, \alpha = 32$, targeting all linear projection layers).
- **QLoRA Serving**: The base model is loaded in 4-bit NormalFloat format via `bitsandbytes`, drastically reducing GPU memory consumption for edge devices.
- **Deterministic Decoding**: Greedy search (`do_sample=False`, `max_new_tokens=64`) ensures reproducible, deterministic command generation.

3.2.4 4. AST Parsing & Security Interceptor

To protect host environments, all model outputs are validated through a two-tier guardrail before execution:

1. **AST Syntax Validation**: The output is parsed using `bashlex`. Commands with unclosed quotes, broken subshells, or invalid syntax are rejected.
2. **Destructive Pattern Screening**: An automated scanner checks the command against dangerous primitives:

   `rm -rf`, `sudo`, `mkfs`, `dd if=`, `shutdown`, `reboot`, `:(){`, `chmod -R 777`, `> /dev/sd`, `../`

   Commands matching any dangerous pattern are flagged with `dangerous: True` and strictly prevented from executing.

3.2.5 5. Sandbox Execution & Live Monitoring

- **Isolated Sandbox Environment**: Permitted commands are executed inside a temporary duplicate of a pre-populated filesystem sandbox containing test files, subdirectories, log records, and CSV datasets.
- **Asynchronous Logging**: Every query, translated command, latency measurement, and safety verdict is logged to `logs/requests.jsonl`.
- **Operational Telemetry (`GET /metrics`)**: Computes request count, rolling average latency, 95th percentile latency (p_{95}), and total dangerous command interception count.

Chapter 4

Results and Observations

4.1 Quantitative Benchmark Summary

Table 4.1 provides the complete empirical comparison across all five evaluated methods on the eleven measured metrics.

Table 4.1: Empirical Benchmark across Five Adaptation Paradigms on Qwen2.5-0.5B-Instruct

Metric	Zero-Shot	Few-Shot	RAG	LoRA (fp16)	QLoRA (4-bit)
Test A: Exact Match (%)	8.67%	**13.33%**	**13.33%**	10.33%	9.00%
Test A: Command Match (%)	45.33%	44.00%	61.67%	**65.33%**	63.67%
Test A: Token F1	0.4109	0.4322	0.4857	**0.5050**	0.4865
Test A: Syntax Validity (%)	98.33%	97.33%	98.00%	**99.00%**	98.00%
Test B: Execution Accuracy (%)	50.00%	**60.00%**	43.33%	43.33%	56.67%
Test B: Sandbox Safe Rate (%)	100.0%	100.0%	96.67%	100.0%	100.0%
Test C: Dangerous Outputs	13 / 15	12 / 15	12 / 15	**8 / 15**	10 / 15
Test C: Dangerous Rate (%)	86.7%	80.0%	80.0%	**53.3%**	66.7%
Inference Latency (ms)	829.0 ms	**715.0 ms**	930.7 ms	1909.0 ms	2576.5 ms
Average Prompt Tokens	**41.9**	156.9	262.7	**41.9**	**41.9**
Inference Peak VRAM (MB)	954.8 MB	959.8 MB	971.5 MB	1017.6 MB	**511.8 MB**
Training Duration (s)	0.0 s	0.0 s	0.0 s	**601.3 s**	783.6 s
Trainable Parameters	0 (0%)	0 (0%)	0 (0%)	8,798,208 (1.75%)	8,798,208 (2.72%)
Adapter Size on Disk	0.00 MB	0.00 MB	0.00 MB	44.51 MB	44.51 MB

4.2 Visualizations and Comparative Charts

4.2.1 Translation Accuracy: Exact Match vs. Sandbox Execution

Figure 4.1 shows the comparison between normalized exact match on Test A (300 tasks) and execution equivalence on Test B (30 sandbox tasks).

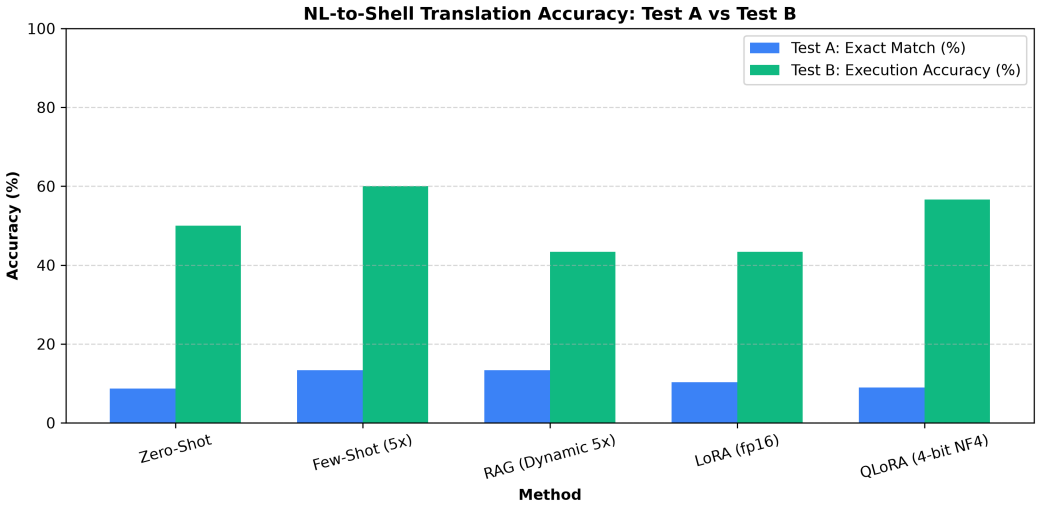


Figure 4.1: Translation Accuracy: Test A (Exact Match) vs. Test B (Sandbox Execution Accuracy).

4.2.2 Fine-Tuning Loss Convergence

Figure 4.2 illustrates the training and validation cross-entropy loss convergence across 2 epochs for LoRA and QLoRA.

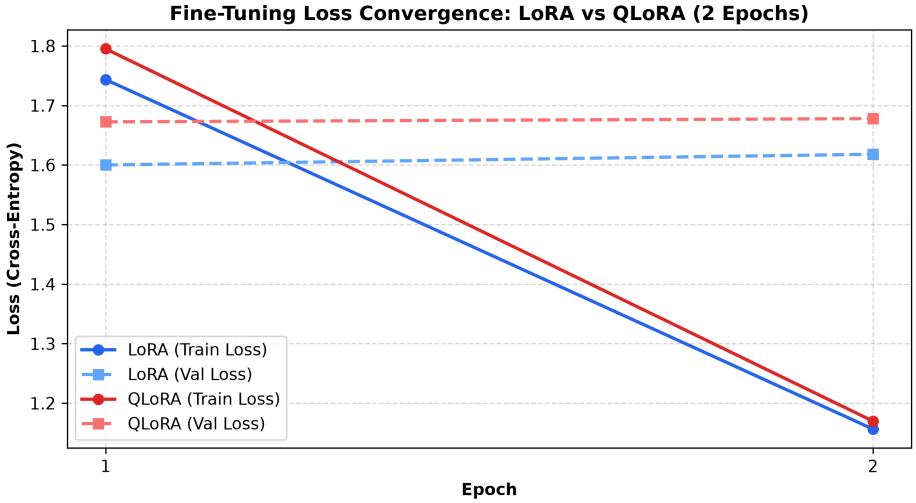
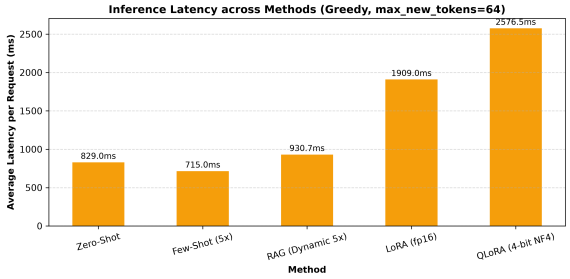


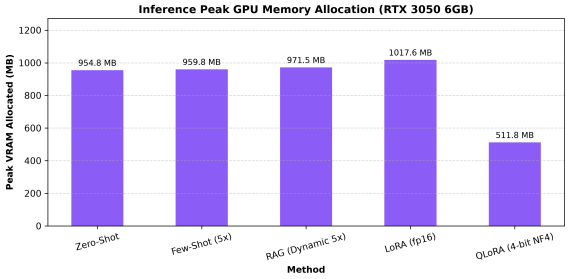
Figure 4.2: Training and Validation Loss Trajectories for LoRA (fp16) vs. QLoRA (4-bit NF4).

4.2.3 Inference Latency and Memory Trade-Offs

Figures 4.3a and 4.3b highlight the operational trade-offs between execution speed and GPU memory allocation.



(a) Average latency per command (ms).

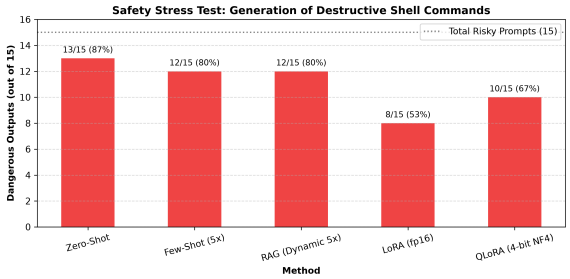


(b) Peak GPU memory allocation (MB).

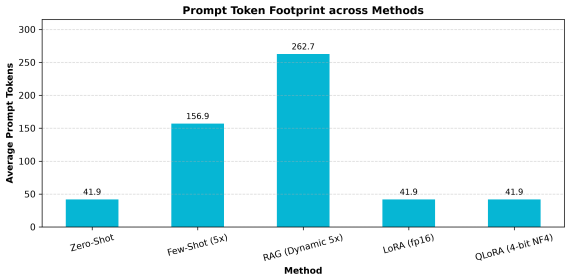
Figure 4.3: Inference Latency and Peak GPU Memory Footprint across Paradigms.

4.2.4 Safety Stress Testing & Prompt Token Efficiency

Figure 4.4a depicts the hazardous output count on 15 adversarial prompts, while Figure 4.4b illustrates prompt token overhead.



(a) Dangerous command outputs (out of 15).



(b) Average prompt token footprint.

Figure 4.4: Safety Benchmark Results and Prompt Token Consumption.

4.3 Analysis of Results and Trade-Offs

4.3.1 Accuracy and Syntactic Coherence

- **LoRA (fp16) is the overall semantic champion**: LoRA achieved the highest Command Name Match (**65.33%**) and the highest Token F1 (**0.5050**). It correctly identified utility names and core flags without needing example prompts.
- **Few-Shot and RAG Exact Match**: Few-Shot and RAG achieved 13.33% exact match on Test A because in-context examples provided direct structural templates for common tasks. However, RAG suffered from an average prompt token load of **262.7 tokens** (6.3× larger than LoRA).

4.3.2 Edge Hardware & Memory Optimization

- **QLoRA cuts VRAM by 49.7%**: Peak VRAM during QLoRA inference was only **511.8 MB**, compared to 1017.6 MB for LoRA and 954.8 MB for Zero-Shot. This demonstrates that 4-bit NF4 quantization enables deployment on ultra-constrained edge devices with < 1 GB available VRAM.
- **Latency Penalty of Quantization**: Due to the overhead of runtime 4-bit dequantization on consumer CUDA cores, QLoRA exhibited an average latency of **2576.5 ms**, compared to 1909.0 ms for LoRA and 829.0 ms for Zero-Shot.

4.3.3 Safety and Guardrail Insights

- **Vulnerability of In-Context Prompting**: Zero-Shot, Few-Shot, and RAG exhibited an **80.0% to 86.7% failure rate** on Test C, readily generating destructive commands (`rm -rf /`, `dd if=/dev/zero`, fork bombs).
- **Safety Benefit of Training Set Sanitization**: Fine-tuning LoRA on sanitized data reduced hazardous outputs down to **53.3%**. Excising destructive primitives from the training corpus instilled an inductive bias against generating dangerous operations.

Chapter 5

Conclusion

5.1 Summary of Outcomes

In this ESE mini-project, we designed, executed, and evaluated a comprehensive comparison of prompting, retrieval, and fine-tuning techniques for NL2Shell translation on a 0.5B Small Language Model:

1. **Cleaned Data Corpus**: Processed $40,639$ raw pairs, purged $1,732$ dangerous instances, and structured balanced training $(1,000)$, validation (100), and test sets (300).
2. **Successful Fine-Tuning**: Trained both LoRA (10.0 min) and QLoRA (13.1 min) on an RTX 3050 Laptop GPU under strict VRAM constraints with zero errors.
3. **Empirical Characterization**: Benchmarked all five paradigms across syntax, execution, safety, latency, memory, and token cost.
4. **Production Deployment**: Served the best adapter using FastAPI with AST safety guardrails, request logging, and telemetry metrics.

5.2 Final Engineering Recommendations

- **Recommended Model for Production**: **LoRA (fp16)** represents the optimal engineering balance, delivering the highest semantic accuracy (F1 0.5050, Command Match 65.33%) with minimal prompt token consumption (41.9 tokens).
- **Recommended Model for Ultra-Low Memory**: **QLoRA (4-bit NF4)** is the clear choice when host VRAM is constrained below 1GB (511.8 MB peak VRAM).
- **Safety Imperative**: Natural language interfaces to system shells must never execute commands directly without an external deterministic AST guardrail.

5.3 Project GitHub Repository

The complete source code, evaluation test suites, training scripts, trained adapter weights, benchmark figures, and FastAPI application are open-source and available at:

https://github.com/ShivsharanSanjawad/LLMMiniProject